Computer Networks

Overview of the Internet
Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
  - World Wide Web and HTTP
  - FTP
  - Electronic Mail
  - DNS
- Peer-to-peer paradigm
  - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

Application Layer

1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

Application Layer

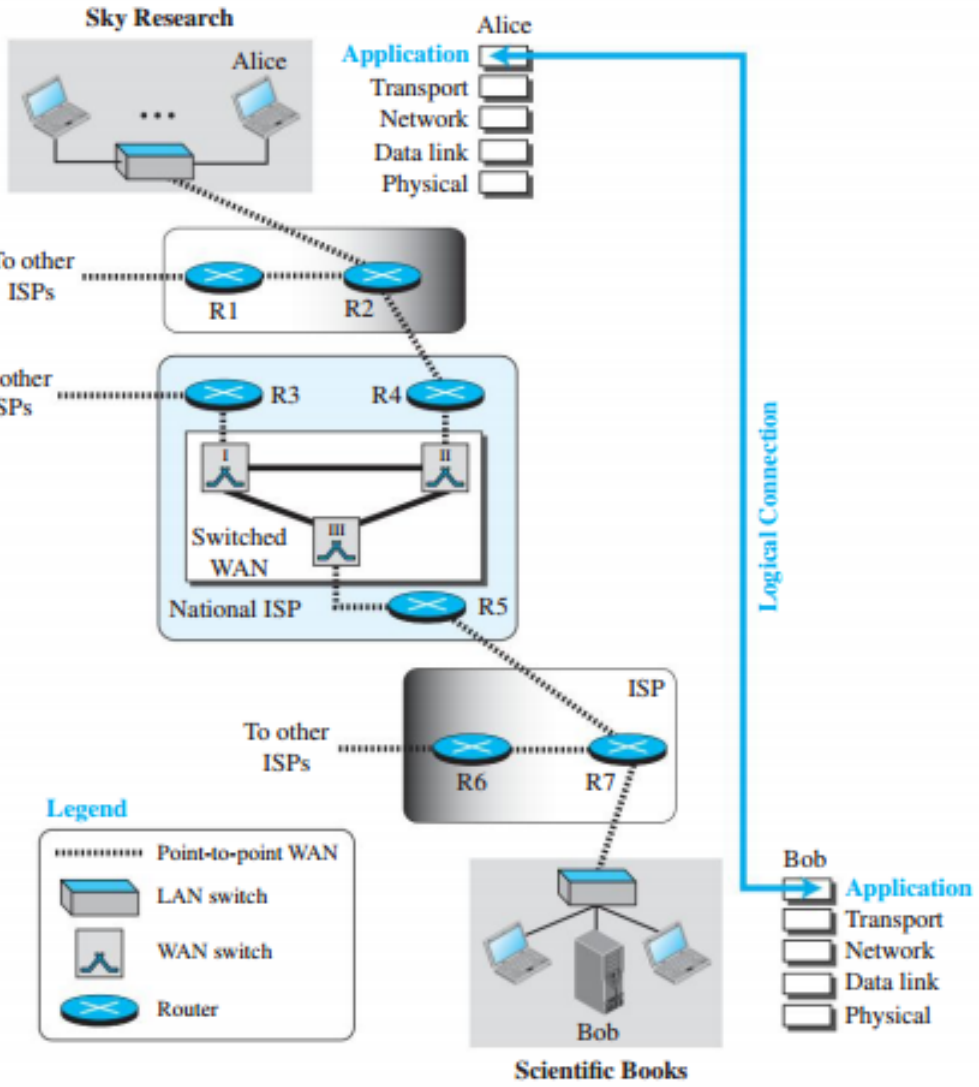
1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

Introduction to the Application Layer

- Application layer provides:
  - Services to the user
- Communication occurs through:
  - Logical connection
- Logical connection means:
  - Imaginary direct communication channel
- Application layers assume:
  - Direct sending and receiving of messages
- Communication is:
  - Logical
  - Not physical

Logical connection at the application layer

Figure 2.1 *Logical connection at the application layer*



Example Scenario of Logical Communication

- Scientist works at:
  - Sky Research
- Scientist needs:
  - Research-related book
- Book ordered from:
  - Scientific Books
- Communication occurs between:
  - Computer at Sky Research
  - Server at Scientific Books
- Hosts are named:
  - Alice
  - Bob

Logical vs Physical Communication

- Alice and Bob assume:
  - Direct two-way logical channel
- Through logical connection they can:
  - Send messages
  - Receive messages
- Actual communication occurs through:
  - Multiple intermediate devices
  - Multiple physical channels
- Communication path includes:
  - Alice
  - R2
  - R4
  - R5
  - R7
  - Bob

Logical Connection Illustration

Alice (Sky Research)

$\Updownarrow$ Logical Communication

Bob (Scientific Books)

Actual Physical Path:

Alice $\rightarrow$ R2 $\rightarrow$ R4 $\rightarrow$ R5 $\rightarrow$ R7 $\rightarrow$ Bob

- Logical communication hides network complexity
- Physical communication uses routers and links

Providing Services

- Communication networks are designed to:
  - Provide services to users
- Early communication networks:
  - Designed for one specific service
- Example:
  - Telephone network designed for voice communication
- Telephone network later supported:
  - Fax communication
- Additional services required:
  - Extra hardware at both ends

Internet as a Service Provider

- Internet designed to:
  - Provide services worldwide
- TCP/IP layered architecture makes Internet:
  - Flexible
- More flexible than:
  - Postal network
  - Telephone network
- Each layer originally contained:
  - One or more protocols
- New protocols can be:
  - Added
  - Removed
  - Replaced

Protocol Flexibility in TCP/IP

- New protocols must:
  - Use services of lower-layer protocols
- Removing a protocol requires:
  - Modification of higher-layer protocols
- Protocol interaction depends on:
  - Layered service structure
- TCP/IP architecture supports:
  - Expandability
  - Interoperability

Special Role of the Application Layer

- Application layer is:
  - Highest layer in TCP/IP suite
- Application-layer protocols:
  - Do not provide services to higher layers
- Application layer only receives:
  - Services from transport layer
- Protocols in this layer can be:
  - Easily added
  - Easily removed
- New application protocols must:
  - Use transport-layer services

Growth of Application Protocols

- Internet originally had:
  - Few application protocols
- Today:
  - Number of application protocols constantly increasing
- Flexibility of application layer allows:
  - Easy development of new applications
- Growth of Internet services driven by:
  - Addition of new application protocols

Standard and Nonstandard Protocols

- Protocols in first four layers must be:
  - Standardized
  - Documented
- Standard protocols included in:
  - Windows
  - UNIX
  - Other operating systems
- Application-layer protocols can be:
  - Standard
  - Nonstandard
- Flexibility mainly exists at:
  - Application layer

Standard Application-Layer Protocols

- Standard protocols are:
  - Approved by Internet authorities
  - Officially documented
- Standard applications provide:
  - Specific Internet services
- Each protocol consists of:
  - Pair of interacting programs
- Programs interact with:
  - User
  - Transport layer
- Understanding these protocols helps:
  - Network management
  - Problem solving
  - Development of new protocols

Nonstandard Application-Layer Protocols

- Programmers can create:
  - Proprietary application protocols
- Requires:
  - Two communicating programs
- Programs use:
  - Services of transport layer
- Private protocols may not require:
  - Approval from Internet authorities
- Companies can develop:
  - Customized communication applications
- New protocols can be written using:
  - Programming languages

Application Layer

1. **Introduction**
   - Providing Services
   - Application-Layer Paradigms
2. **Standard Client-Server Applications**
   - World Wide Web and HTTP
   - FTP
   - Electronic Mail
   - Domain Name System (DNS)
3. **Peer-to-Peer Paradigm**
   - P2P Networks

Application-Layer Paradigms

- Internet communication requires:
  - Two interacting application programs
- Programs run on:
  - Different computers
  - Different locations in the world
- Programs exchange:
  - Messages through Internet infrastructure
- Important question:
  - Should programs both request and provide services?
- Two major paradigms:
  - Client-Server Paradigm
  - Peer-to-Peer Paradigm

Traditional Paradigm: Client-Server

- Traditional Internet model:
  - Client-Server Paradigm
- Service provider:
  - Server process
- Service requester:
  - Client process
- Server process:
  - Runs continuously
  - Waits for client requests
- Client process:
  - Starts only when service is needed

Characteristics of Client-Server Paradigm

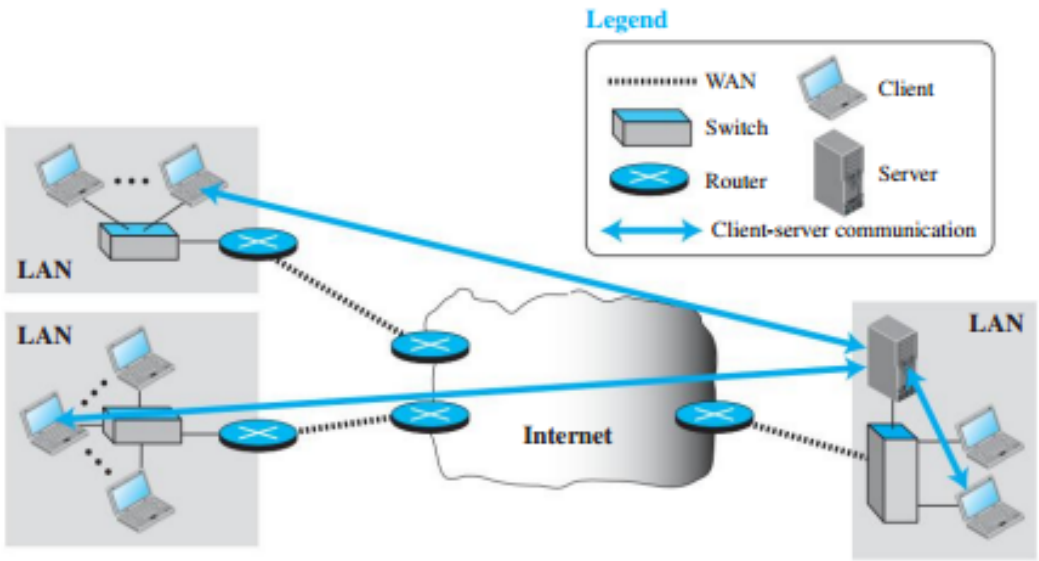
- Multiple clients can access:
  - Same server
- Server provides:
  - Specific type of service
- Communication roles are:
  - Different
  - Fixed
- Client program cannot act as:
  - Server program
- Separate programs required for:
  - Client
  - Server

Example of Client-Server Communication

- Example:
  - Telephone directory center
- Directory center acts as:
  - Server
- Subscriber acts as:
  - Client
- Directory center must:
  - Remain available continuously
- Subscriber connects:
  - Only when service is needed

Example of a client-server paradigm

Figure 2.2 *Example of a client-server paradigm*



Client-Server Architecture Illustration

Client 1 →
Client 2 → Server
Client 3 →

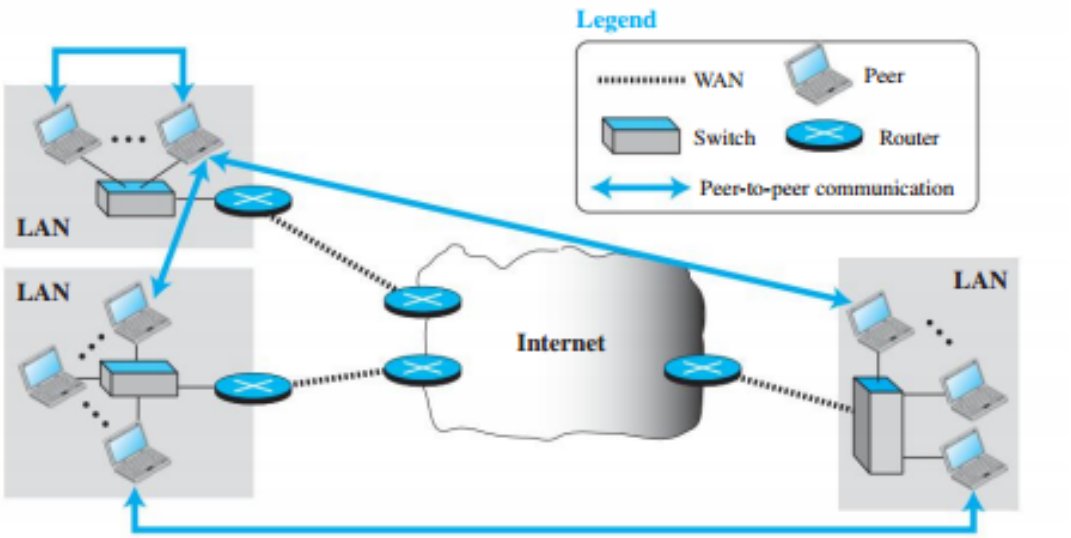
- One server provides service to multiple clients
- Server centrally controls communication

Peer-to-Peer Paradigm

- Alternative communication model:
  - Peer-to-Peer (P2P) Paradigm
- In P2P:
  - Each host can provide services
  - Each host can request services
- No dedicated server required
- Peers communicate:
  - Directly with each other
- Every peer can act as:
  - Client
  - Server

Example of a peer-to-peer paradigm

Figure 2.3 *Example of a peer-to-peer paradigm*



Advantages of Peer-to-Peer Paradigm

- P2P paradigm is:
  - Easily scalable
  - Cost-effective
- Eliminates need for:
  - Expensive dedicated servers
- No need for server to:
  - Run continuously
- Suitable for:
  - File sharing
  - Distributed communication

Challenges in Peer-to-Peer Paradigm

- Main challenge:
  - Security
- Secure communication more difficult:
  - In distributed systems
- Another challenge:
  - Applicability
- Not all Internet applications:
  - Suitable for P2P model
- Example:
  - World Wide Web difficult to fully implement as P2P

Applications Using Peer-to-Peer Paradigm

- Popular P2P applications:
  - BitTorrent
  - Skype
  - IPTV
  - Internet telephony
- P2P supports:
  - Distributed resource sharing
  - Direct communication between users

Mixed Paradigm

- Some applications combine:
  - Client-Server Paradigm
  - Peer-to-Peer Paradigm
- Mixed paradigm uses:
  - Advantages of both approaches
- Example:
  - Client-server used to locate peers
  - P2P used for actual communication
- Hybrid approach improves:
  - Scalability
  - Efficiency

Comparison of Paradigms

Feature	**Client-Server**	**Peer-to-Peer**
Dedicated Server	Yes	No
Scalability	Moderate	High
Cost	High	Lower
Security	Easier	More Difficult
Communication	Centralized	Distributed
Examples	Web, E-mail	BitTorrent, Skype

Client-Server Paradigm

- Communication occurs between:
  - Client process
  - Server process
- Client:
  - Initiates communication
  - Sends request
- Server:
  - Waits for requests
  - Processes requests
  - Sends responses
- Communication occurs at:
  - Application layer

Lifetime of Client and Server

- Server process:
  - Must run continuously
  - Waits forever for clients
- Lifetime of server:
  - Infinite
- Client process:
  - Runs only when needed
  - Sends finite number of requests
- Lifetime of client:
  - Finite
- Server must start:
  - Before client

Application Programming Interface (API)

- Client-server communication requires:
  - Special communication instructions
- API stands for:
  - Application Programming Interface
- API provides:
  - Interface between application and operating system
- API allows processes to:
  - Open connections
  - Send data
  - Receive data
  - Close connections

Role of the Operating System

- Operating system contains:
  - First four layers of TCP/IP suite
- Application layer communicates with:
  - Operating system through API
- API acts as:
  - Communication bridge
- Operating system responsible for:
  - Network communication handling

Common Communication APIs

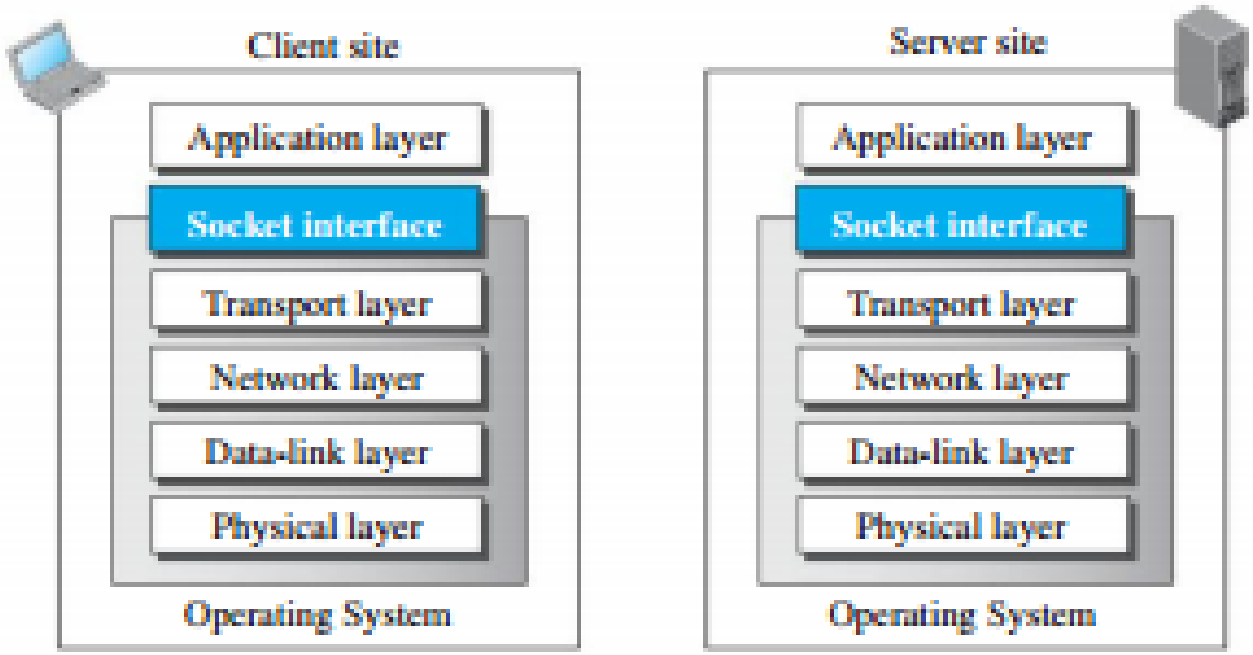
- Common APIs include:
  - Socket Interface
  - Transport Layer Interface (TLI)
  - STREAM
- Most popular API:
  - Socket Interface
- Socket interface introduced:
  - Early 1980s
  - UC Berkeley
- Originally part of:
  - UNIX environment

Socket Interface

- Socket interface provides:
  - Communication between application and operating system
- Socket interface allows:
  - Process-to-process communication
- Sockets behave like:
  - Files
  - Input/output devices
- Existing programming instructions can:
  - Read from sockets
  - Write to sockets

Position of the socket interface

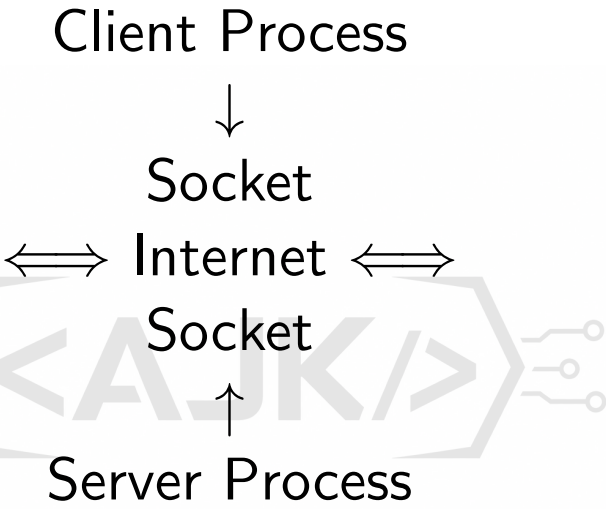
Figure 2.4 *Position of the socket interface*



Sockets as Abstractions

- Socket is:
  - Not a physical entity
- Socket is:
  - Abstract data structure
- Created and used by:
  - Application programs
- Communication between client and server:
  - Communication between two sockets
- Operating system and TCP/IP handle:
  - Actual data transfer

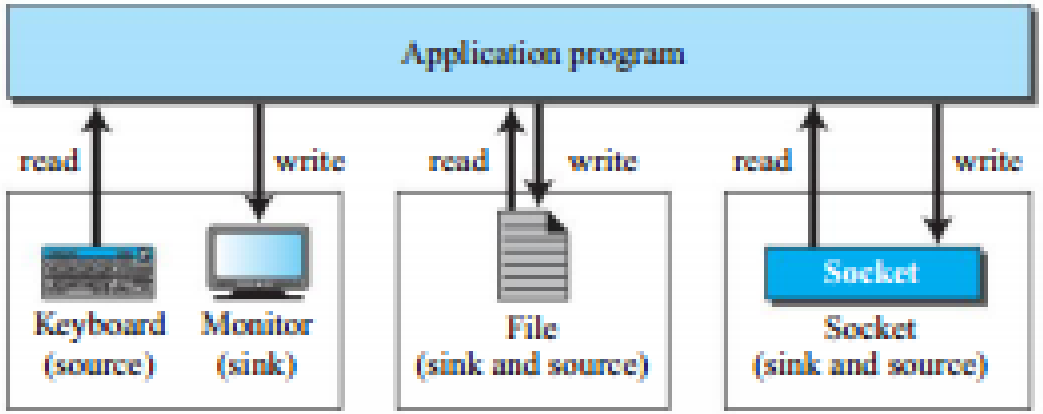
Socket Communication Model



- Communication occurs through socket pair
- Sockets act as endpoints of communication

Sockets used the same way as other sources and sinks

Figure 2.5 *Sockets used the same way as other sources and sinks*

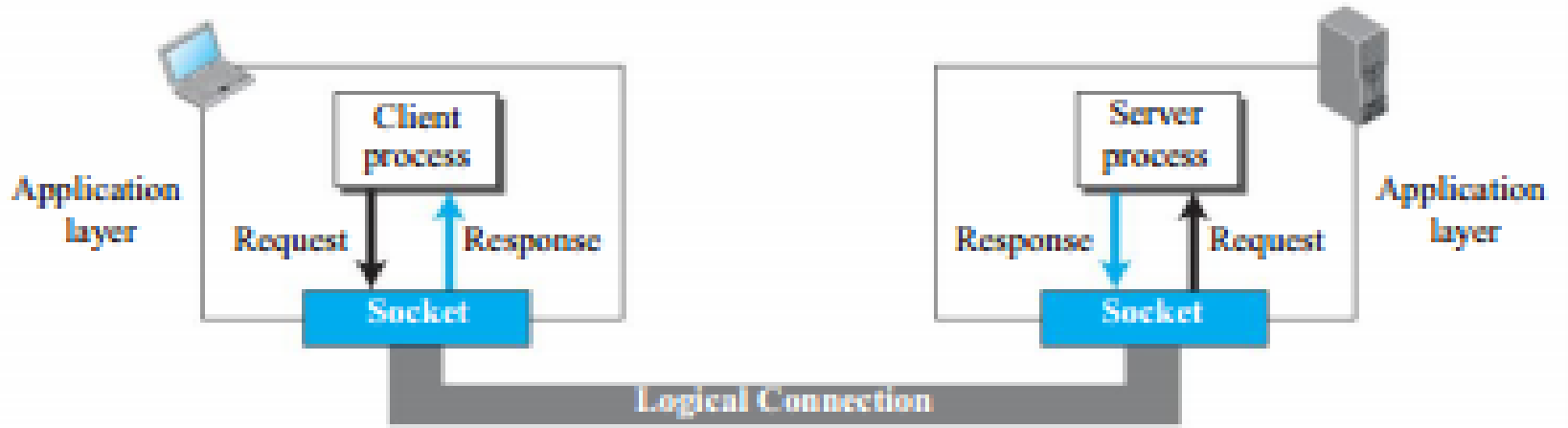


Socket Addresses

- Two-way communication requires:
  - Local address
  - Remote address
- Communication between sockets requires:
  - Pair of socket addresses
- Socket address consists of:
  - IP address
  - Port number
- IP address:
  - Identifies computer
- Port number:
  - Identifies application process

Use of sockets in process-to-process communication

Figure 2.6 *Use of sockets in process-to-process communication*



Socket Address Structure

$$\text{Socket Address} = \text{IP Address} + \text{Port Number}$$

- IP address:
  - 32-bit identifier
- Port number:
  - 16-bit identifier
- Socket uniquely identifies:
  - Communication endpoint

A socket address

Figure 2.7 *A socket address*

Telephone Analogy for Socket Addressing

- Telephone communication uses:
  - Telephone number
  - Extension number
- Telephone number similar to:
  - IP address
- Extension number similar to:
  - Port number
- Combination identifies:
  - Specific connection

Server Socket Addresses

- Server requires:
  - Local socket address
  - Remote socket address
- Local socket address:
  - Fixed during server lifetime
- Standard servers use:
  - Well-known port numbers
- Example:
  - HTTP uses port 80
- Remote socket address:
  - Obtained from client request

Client Socket Addresses

- Client also requires:
  - Local socket address
  - Remote socket address
- Local client port:
  - Temporary
  - Ephemeral port number
- Operating system ensures:
  - Port number uniqueness
- Remote socket address:
  - Server IP address
  - Server port number

Finding Remote Server Address

- Client may directly know:
  - Server IP address
  - Server port number
- Common Internet applications use:
  - Server names
  - URLs
  - E-mail addresses
- DNS converts:
  - Server name to IP address
- DNS acts like:
  - Internet telephone directory